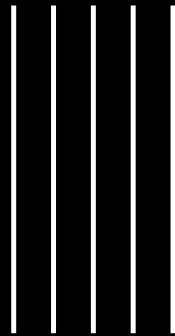


Simulasi Realistis Git & GitHub Team Workflow



Afrizal F.A

Simulasi Realistis Git & GitHub Team Workflow

Latihan tim 1–4 akun: Pull Request, review, CI, conflict,
squash merge, hotfix, parallel work, dan cherry-pick

Afrizal F.A

2026

Kata Pengantar

Buku ini disusun sebagai modul praktik Git & GitHub dengan format simulasi tim yang realistis: ada Tech Lead, developer, reviewer, ticket, Pull Request, CI, konflik, hotfix, sampai cherry-pick ke release branch. Semua nama perusahaan, akun, dan ticket adalah fiksi untuk latihan.

Semua file di buku ini diberikan UTUH dalam blok perintah `"cat > file << 'EOF'"` yang tinggal di-copy-paste ke terminal. Jangan mengetik ulang atau menyalin sebagian file – itu sumber error paling umum (module.exports hilang, indentasi YAML rusak, dsb).

Penjelasan sengaja singkat dan to the point. Kerjakan bab secara berurutan, karena setiap bab dibangun di atas bab sebelumnya. Selamat berlatih.

Daftar Isi

BAB 1 – Pendahuluan

BAB 2 – Persiapan: SSH Multi-Akun dan Repository Awal

BAB 3 – Task Pertama: APP-101

BAB 4 – Review, CI, dan Revisi

BAB 5 – Konflik dan Integrasi

BAB 6 – Squash Merge, Sinkronisasi, dan Hotfix

BAB 7 – Simulasi 3 Akun: Parallel Work

BAB 8 – Simulasi 4 Akun: Dua Reviewer

BAB 9 – Cherry-pick

BAB 10 – Penutup

BAB 1 – Pendahuluan

Kamu bergabung ke tim fiksi Acme App Team yang mengelola repository `acme-profile-app`: aplikasi profil sederhana berbasis JavaScript. Lewat repository ini kamu menjalani siklus kerja tim software modern dari awal sampai selesai.

```
main <- Pull Request <- feature branch <- commits lokal
```

Empat mode latihan

Mode	Akun	Yang dilatih
Pemanasan	1	Branch, commit, push, PR, merge
Realistis	2	Protected main, request changes, approval
Tim asli	3	Parallel work, review silang, konflik alami
Perusahaan	4	Dua reviewer, CODEOWNERS, approval ganda

Pembagian role

Akun	Role	Tanggung jawab
A	Tech Lead / Maintainer	Repo, aturan main, ticket, review akhir, merge
B	Developer 1 / Reviewer 1	Coding, PR, me-review PR orang lain
C	Developer 2 / Reviewer 2	Coding paralel, review silang (mode 3-4)
D	Developer (author)	Mengerjakan task, menangani feedback (mode 4)

Prinsip utama buku ini: developer tidak pernah bekerja langsung di main. Semua perubahan masuk lewat branch dan Pull Request yang direview.

BAB 2 – Persiapan: SSH Multi-Akun dan Repository Awal

Setup Git dasar

```
git --version
git config --global user.name "Nama Kamu"
git config --global user.email "email-kamu"
```

Multi-akun di satu PC lewat SSH

Untuk mode 2-4 akun di satu komputer: satu SSH key per akun, lalu daftarkan alias per akun di ~/.ssh/config. Nama di bawah ini contoh fiksi – ganti dengan nama akun latihanmu sendiri.

```
# 1) Buat satu key per akun (email = email akun GitHub tsb)
ssh-keygen -t ed25519 -C "lead@example.com" \
-f ~/.ssh/id_ed25519_acme_lead
ssh-keygen -t ed25519 -C "dev1@example.com" \
-f ~/.ssh/id_ed25519_acme_dev1
ssh-keygen -t ed25519 -C "dev2@example.com" \
-f ~/.ssh/id_ed25519_acme_dev2
ssh-keygen -t ed25519 -C "dev3@example.com" \
-f ~/.ssh/id_ed25519_acme_dev3
```

```
# 2) Daftarkan alias per akun (copy-paste utuh)
cat >> ~/.ssh/config << 'EOF'
Host acme-lead
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_lead
  IdentitiesOnly yes

Host acme-dev1
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_dev1
  IdentitiesOnly yes

Host acme-dev2
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_dev2
  IdentitiesOnly yes

Host acme-dev3
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_dev3
  IdentitiesOnly yes
EOF
```



```
# 3) Upload public key ke akun GitHub masing-masing
# (Settings -> SSH and GPG keys -> New SSH key)
cat ~/.ssh/id_ed25519_acme_lead.pub
cat ~/.ssh/id_ed25519_acme_dev1.pub
cat ~/.ssh/id_ed25519_acme_dev2.pub
cat ~/.ssh/id_ed25519_acme_dev3.pub

# 4) Test: harus menyapa username akun yang sesuai
ssh -T acme-lead
ssh -T acme-dev1

# (optional) simpan passphrase agar tidak ditanya terus
ssh-add ~/.ssh/id_ed25519_acme_dev1
```

Aturan pakainya: saat clone/remote, GANTI git@github.com dengan alias akunnya. Satu folder clone = satu akun = satu identitas.

```
# clone per akun via alias (folder terpisah per role)
git clone acme-lead:<user-A>/acme-profile-app.git team-work-a
git clone acme-dev1:<user-A>/acme-profile-app.git team-work-b
git clone acme-dev2:<user-A>/acme-profile-app.git team-work-c
git clone acme-dev3:<user-A>/acme-profile-app.git team-work-d

# set identitas commit PER folder (tanpa --global)
cd team-work-b
git config user.name "Acme Dev One"
git config user.email "dev1@example.com"

# repo terlanjur di-clone pakai github.com? ganti remote-nya:
git remote set-url origin acme-dev1:<user-A>/acme-profile-app.git
git remote -v
```

Project awal (Akun A – Tech Lead)

Buat repo kosong acme-profile-app di GitHub (tanpa README), lalu jalankan blok-blok berikut APA ADANYA. Semua file dibuat lengkap lewat heredoc supaya tidak ada yang terlewat.

```
mkdir acme-profile-app && cd acme-profile-app
mkdir -p src test .github/workflows
```

```
cat > package.json << 'EOF'
{
  "name": "acme-profile-app",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "test": "node --test"
  }
}
EOF
```

package.json ini WAJIB. Tanpa script "test": "node --test", perintah npm test akan gagal dengan pesan "Error: no test specified".

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio) {
  return { displayName, bio };
}

module.exports = { createProfile };
EOF
```

```
cat > test/profile.test.js << 'EOF'
// test/profile.test.js (node:test, tanpa dependency)
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});
EOF
```

```
cat > .github/workflows/ci.yml << 'EOF'
name: ci
on: [pull_request, push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm test
EOF
```

YAML sensitif terhadap indentasi. Kalau kamu mengetik ulang dan spasinya bergeser, CI akan gagal parse – selalu pakai blok copy-paste di atas.

```
cat > .gitignore << 'EOF'
node_modules/
EOF
```

```
# pastikan test lulus SEBELUM commit pertama
npm test

git init
git add .
git commit -m "chore: initialize profile app"
git branch -M main
git remote add origin acme-lead:<user-A>/acme-profile-app.git
git push -u origin main
```

Aturan branch main

Aturan	Tujuan
Require Pull Request	Tidak ada direct push ke main
Require approval	Minimal 1 reviewer setuju (mode 4: minimal 2)
Require status checks	CI harus hijau sebelum merge
Block force push	History main tidak ditulis ulang

Tambahkan akun B (dan C, D sesuai mode) sebagai collaborator. Jika fitur protection tidak tersedia di plan repositorymu, jalankan aturannya secara disiplin manual.

BAB 3 – Task Pertama: APP-101

Ticket dari Tech Lead

```
APP-101 – Add editable user profile
- displayName dan bio tersimpan
- displayName wajib diisi
- Spasi depan/belakang dibersihkan
- Test lulus
Out of scope: avatar, database, auth
```

Urutan yang benar: branch DULU, baru coding

Kesalahan paling umum: coding dan commit dulu di main, lalu push feature branch yang belum ada – hasilnya error "src refs spec ... does not match any". Selalu buat branch sebelum menyentuh code, dan cek posisi branch sebelum setiap commit.

```
# Akun B, di folder team-work-b
git switch main
git pull origin main
git switch -c feature/APP-101-editable-profile

# WAJIB sebelum commit apa pun – harus tercetak nama feature branch
git branch --show-current
# feature/APP-101-editable-profile
```

Kalau terlanjur salah branch

```
# terlanjur EDIT di main (belum commit): bawa perubahan pindah
git switch -c feature/APP-101-editable-profile

# terlanjur COMMIT di main (belum push): pindahkan commit
git branch feature/APP-101-editable-profile
git reset --hard origin/main
git switch feature/APP-101-editable-profile
```

Commit 1 – normalisasi (file UTUH, copy-paste)

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio = "") {
  return {
    displayName: displayName.trim(),
    bio: bio.trim(),
  };
}

module.exports = { createProfile };
EOF
```

Perhatikan: baris `module.exports` SELALU ada di akhir file. Kalau hilang, test langsung gagal dengan "createProfile is not a

function".

```
git branch --show-current # feature/APP-101-editable-profile
git status && git diff
npm test # semua PASS dulu, baru commit
git add src/profile.js
git commit -m "feat: normalize profile fields"
```

Commit 2 – validation + test (file UTUH, copy-paste)

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio = "") {
  const cleanName = displayName.trim();
  const cleanBio = bio.trim();

  if (!cleanName) {
    throw new Error("displayName is required");
  }

  return { displayName: cleanName, bio: cleanBio };
}

module.exports = { createProfile };
EOF
```

```
cat > test/profile.test.js << 'EOF'
// test/profile.test.js
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});

test("trims whitespace", () => {
  assert.equal(createProfile(" Raka ").displayName, "Raka");
});

test("rejects empty displayName", () => {
  assert.throws(() => createProfile(" "), /required/);
});
EOF
```

```
git branch --show-current # feature/APP-101-editable-profile
npm test
git add src/profile.js test/profile.test.js
git commit -m "feat: validate required display name"
```

Push dan buka Pull Request

```
git push -u origin feature/APP-101-editable-profile
```

```
Title : APP-101: add editable user profile validation
Summary : normalize + validate displayName/bio, add tests
Test : npm test; " Raka " -> "Raka"; nama kosong -> error
Closes APP-101
```

Sebelum minta review, lakukan self-review di tab Files changed: cari debug code, file nyasar, secret, atau perubahan di luar scope.

Error umum di fase awal (dan obatnya)

Error	Penyebab	Solusi
Error: no test specified	package.json tidak ada / script test kosong	Buat package.json persis seperti Bab 2
createProfile is not a function	module.exports terhapus saat edit	Tulis file utuh dari blok copy-paste
displayName is not defined	Menyalin potongan code, bukan file utuh	Selalu pakai versi file lengkap
src refs spec ... does not match any	Branch belum dibuat; commit nyasar di main	git switch -c dulu; cek git branch --show-current
CI gagal parse YAML	Indentasi rusak saat mengetik ulang	Copy-paste blok ci.yml apa adanya

BAB 4 – Review, CI, dan Revisi

Checks otomatis

```
Pull Request #14  
[PASS] ci / test  
[WAIT] review
```

CI merah? Jangan minta reviewer mengabaikannya. Baca log, perbaiki di branch yang sama, push – PR terupdate otomatis.

Request changes (Akun A)

```
src/profile.js  
"displayName.trim() error bila input bukan string.  
Tangani input invalid secara eksplisit + test."
```

```
Review result: REQUEST CHANGES
```

Request changes itu normal, bukan kegagalan. Jangan buat PR baru – perbaiki di branch yang sama.

Menangani review (Akun B) – file UTUH, copy-paste

```
git switch feature/APP-101-editable-profile  
git branch --show-current # pastikan benar
```

```
cat > src/profile.js << 'EOF'  
// src/profile.js  
function createProfile(displayName, bio = "") {  
  if (typeof displayName !== "string") {  
    throw new Error("displayName must be a string");  
  }  
  
  const cleanName = displayName.trim();  
  const cleanBio = typeof bio === "string" ? bio.trim() : "";  
  
  if (!cleanName) {  
    throw new Error("displayName is required");  
  }  
  
  return { displayName: cleanName, bio: cleanBio };  
}  
  
module.exports = { createProfile };  
EOF
```

```
cat > test/profile.test.js << 'EOF'
// test/profile.test.js
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});

test("trims whitespace", () => {
  assert.equal(createProfile("  Raka  ").displayName, "Raka");
});

test("rejects empty displayName", () => {
  assert.throws(() => createProfile("  "), /required/);
});

test("rejects non-string displayName", () => {
  assert.throws(() => createProfile(123), /must be a string/);
  assert.throws(() => createProfile(null), /must be a string/);
});
EOF
```

```
npm test
git add src/profile.js test/profile.test.js
git commit -m "fix: handle invalid profile input"
git push
```

Balas komentar review dengan singkat, misalnya "Handled invalid input + added coverage", lalu minta re-review.

BAB 5 – Konflik dan Integrasi

Main bergerak saat PR direview

```
A---B---E---F main          (maju duluan)
      \---C---D feature/APP-101 (tertinggal)
```

```
git fetch origin
git log --oneline --graph --decorate --all
```

Merge vs rebase

Cara	Efek	Catatan
git merge origin/main	History asli + merge commit	Aman untuk branch yang sudah dipush
git rebase origin/main	History linear, commit ditulis ulang	Butuh force push; ikuti policy tim

Membaca conflict marker

```
<<<<<<< HEAD
  return { displayName: cleanName, bio: cleanBio };
=====
  return { displayName: cleanName, bio: cleanBio,
            updatedAt: Date.now() };
>>>>>> origin/main
```

- Saat MERGE: HEAD = branch feature-mu; sisi bawah = origin/main.
- Saat REBASE: terbalik - HEAD = main (base baru), sisi bawah = commit feature-mu yang sedang direplay.
- Jangan asal hapus marker: pahami maksud kedua sisi, gabungkan, dan pastikan file kembali UTUH dan valid (module.exports tetap ada), lalu test ulang.

Menyelesaikan

```
// hasil gabungan kedua maksud (di dalam createProfile)
return { displayName: cleanName, bio: cleanBio,
        updatedAt: Date.now() };
```

```
npm test
# jika merge:
git add src/profile.js && git commit && git push
# jika rebase:
git add src/profile.js && git rebase --continue
git push --force-with-lease --force-if-includes
```

Jangan pernah git push --force polos. --force-with-lease menolak push bila remote berubah tanpa sepengetahuanmu.

BAB 6 – Squash Merge, Sinkronisasi, dan Hotfix

Squash and merge (Akun A)

```
feature: 3 commit kecil + resolusi conflict
main   : 1 commit ->
"APP-101: add editable user profile validation (#14)"
```

Syarat merge: checks hijau, approval lengkap, conversations resolved, conflict selesai.

Setelah merge (Akun B)

```
git switch main && git pull origin main
git branch -d feature/APP-101-editable-profile
git push origin --delete feature/APP-101-editable-profile
```

Hotfix APP-102 – batasi displayName 50 karakter

```
git switch main && git pull origin main
git switch -c fix/APP-102-display-name-length
git branch --show-current # fix/APP-102-display-name-length
```

```
cat > src/profile.js << 'EOF'
// src/profile.js
const MAX_NAME = 50;

function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }
  if (cleanName.length > MAX_NAME) {
    throw new Error("displayName max 50 characters");
  }

  return { displayName: cleanName, bio: cleanBio };
}

module.exports = { createProfile };
EOF
```

```

cat > test/profile.test.js << 'EOF'
// test/profile.test.js
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});

test("trims whitespace", () => {
  assert.equal(createProfile("  Raka  ").displayName, "Raka");
});

test("rejects empty displayName", () => {
  assert.throws(() => createProfile("  "), /required/);
});

test("rejects non-string displayName", () => {
  assert.throws(() => createProfile(123), /must be a string/);
});

test("boundary 50 ok, 51 ditolak", () => {
  assert.ok(createProfile("a".repeat(50)));
  assert.throws(() => createProfile("a".repeat(51)), /max 50/);
});
EOF

```

```

npm test
git add src/profile.js test/profile.test.js
git commit -m "fix: limit profile display name length"
git push -u origin fix/APP-102-display-name-length

```

Buka PR terpisah. Hotfix harus kecil, satu scope, mudah direvert.
 Cek boundary: 50 diterima, 51 ditolak.

BAB 7 – Simulasi 3 Akun: Parallel Work

Mode ini melatih hal yang tidak bisa disimulasikan 2 akun: dua developer bekerja bersamaan di file yang sama, review silang, dan konflik yang muncul alami.

Setup (Akun A)

- Tambahkan B dan C sebagai collaborator; protection: PR + 1 approval + CI.
- Buat dua ticket yang sengaja menyentuh src/profile.js:

```
APP-103 (B): tambah field opsional "location" (trim, default "")
APP-104 (C): tambah formatProfile(profile) -> "name - bio"
```

Akun B - APP-103 (file UTUH, copy-paste)

```
# di folder team-work-b
git switch main && git pull origin main
git switch -c feature/APP-103-location-field
git branch --show-current
```

```
cat > src/profile.js << 'EOF'
// src/profile.js
const MAX_NAME = 50;

function createProfile(displayName, bio = "", location = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }
  if (typeof location !== "string") {
    throw new Error("location must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";
  const cleanLocation = location.trim();

  if (!cleanName) {
    throw new Error("displayName is required");
  }
  if (cleanName.length > MAX_NAME) {
    throw new Error("displayName max 50 characters");
  }

  return {
    displayName: cleanName,
    bio: cleanBio,
    location: cleanLocation,
  };
}

module.exports = { createProfile };
EOF
```

```
npm test
git add src/profile.js
git commit -m "feat: add optional location field"
git push -u origin feature/APP-103-location-field
```

Akun C – APP-104, dikerjakan BERSAMAAN (file UTUH)

```
# di folder team-work-c
git switch main && git pull origin main
git switch -c feature/APP-104-profile-formatter
git branch --show-current
```

```
cat > src/profile.js << 'EOF'
// src/profile.js
const MAX_NAME = 50;

function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }
  if (cleanName.length > MAX_NAME) {
    throw new Error("displayName max 50 characters");
  }

  return { displayName: cleanName, bio: cleanBio };
}

function formatProfile(profile) {
  if (!profile.bio) {
    return profile.displayName;
  }
  return profile.displayName + " - " + profile.bio;
}

module.exports = { createProfile, formatProfile };
EOF
```

```
npm test
git add src/profile.js
git commit -m "feat: add profile formatter"
git push -u origin feature/APP-104-profile-formatter
```

Naskah role-play

#	Akun	Aksi
1	B + C	Branch + coding + push PR masing-masing (bersamaan)
2	B <-> C	Review silang: B me-review PR C, C me-review PR B
3	A	Approve + squash merge PR yang siap duluan (misal APP-103)
4	C	PR-nya tertinggal: fetch + merge origin/main, resolve conflict, test, push
5	A	Review ulang, approve, squash merge APP-104
6	B, C	Sync main, hapus branch

Konflik alami yang dialami C

```
git fetch origin
git merge origin/main
```

```
<<<<<< HEAD
module.exports = { createProfile, formatProfile };
=====
module.exports = { createProfile };
>>>>>> origin/main
```

Resolusi: pertahankan maksud KEDUA sisi – fungsi createProfile memakai versi main (yang sudah punya location), dan baris exports tetap memuat formatProfile:

```
module.exports = { createProfile, formatProfile };
```

```
npm test
git add src/profile.js
git commit
git push
```

Aturan mode paralel

- Jangan push ke branch orang lain; beri feedback lewat komentar PR.
- Yang merge belakangan wajib integrasi + test ulang.
- Konflik besar berulang = tanda task perlu dipecah atau butuh koordinasi sebelum coding.

BAB 8 – Simulasi 4 Akun: Dua Reviewer

Mode paling mendekati perusahaan: satu author, dua reviewer wajib, dan maintainer yang mengeksekusi merge.

Role

Akun	Role
A	Tech Lead / Maintainer – aturan, keputusan akhir, merge
B	Reviewer 1 (senior dev)
C	Reviewer 2 (senior dev)
D	Developer – author PR

Setup tambahan (Akun A)

- Require approvals: 2.
- Aktifkan "dismiss stale approvals": push baru menggugurkan approval lama.
- CODEOWNERS agar B dan C otomatis diminta review:

```
cat > .github/CODEOWNERS << 'EOF'
src/  @akun-b @akun-c
EOF
```

Naskah role-play

#	Akun	Aksi
1	A	Buat ticket APP-105: fix empty bio handling; assign ke D
2	D	Branch fix/APP-105-empty-bio, coding + test, push, buka PR
3	B	Review: REQUEST CHANGES (minta test tambahan)
4	C	Review: APPROVE – dua reviewer boleh beda pendapat
5	D	Perbaiki di branch yang sama, push
6	–	Approval C otomatis gugur (stale) karena ada commit baru
7	B	Re-review: APPROVE

#	Akun	Aksi
8	C	Re-review: APPROVE (2/2 lengkap)
9	A	Cek checks + conversations, lalu squash merge
10	D	Sync main, hapus branch

Command Akun D (copy-paste)

```
# di folder team-work-d
git switch main && git pull origin main
git switch -c fix/APP-105-empty-bio
git branch --show-current
# ... edit sesuai ticket (file utuh!), lalu:
npm test
git add src/profile.js test/profile.test.js
git commit -m "fix: handle empty bio consistently"
git push -u origin fix/APP-105-empty-bio
```

```
Checks    [PASS] ci / test
Reviews   [PASS] 2/2 approvals (B, C)
Status    READY TO MERGE
```

Dinamika dua reviewer

- Reviewer beda pendapat? Diskusikan di thread PR sampai ada keputusan, bukan lewat jalur pribadi. Tech Lead jadi pemutus bila buntu.
- Author tidak pernah meng-approve PR-nya sendiri.
- 2 approval bukan auto-merge - merge tetap keputusan maintainer.
- Commit baru = review ulang. Itu sebabnya PR kecil lebih cepat masuk.

BAB 9 – Cherry-pick

Apa itu

Cherry-pick menyalin SATU commit tertentu ke branch lain sebagai commit baru (SHA berbeda, isi perubahan sama). Berguna saat kamu butuh satu perbaikan tanpa membawa seluruh branch.

Kapan dipakai

- Membawa hotfix dari main ke release branch (kasus paling umum).
- Menyelamatkan commit yang terlanjur dibuat di branch yang salah.
- Mengambil sebagian kecil pekerjaan tanpa merge semuanya.

Command inti

```
git log --oneline main          # cari SHA commit target
git switch release/1.0
git cherry-pick -x <sha>        # -x catat asal commit
git cherry-pick -x <sha1> <sha2> # beberapa sekaligus
git cherry-pick --continue      # setelah resolve conflict
git cherry-pick --abort         # batalkan
```

Simulasi: bawa hotfix APP-102 ke release/1.0

```
# Akun A membuat release branch (sekali saja)
git switch main && git pull
git switch -c release/1.0 && git push -u origin release/1.0

# Developer membawa hotfix
git switch main && git pull origin main
git log --oneline -5 # catat SHA commit hotfix APP-102
git switch release/1.0
git pull origin release/1.0
git cherry-pick -x <sha-hotfix>
npm test
git push origin release/1.0
# atau lewat PR bila release branch juga protected
```

Konflik saat cherry-pick diselesaikan persis seperti biasa: edit file, git add, lalu git cherry-pick --continue.

Aturan keselamatan

- Cherry-pick = duplikasi commit; bukan pengganti merge untuk pekerjaan besar.
- Selalu pakai -x agar jejak asal commit tercatat.
- Test ulang setelah pick - konteks branch target bisa berbeda.

- Rantai cherry-pick panjang antar branch aktif = resep konflik; pertimbangkan merge.

BAB 10 – Penutup

Workflow ringkas

```
MULAI : git switch main -> git pull -> switch -c feature/T-x
CEK   : git branch --show-current SEBELUM commit apa pun
KERJA : edit file UTUH -> npm test -> status/diff -> add -> commit
KIRIM : git push -u origin <branch> -> buka PR -> self-review
REVIEW : request changes? perbaiki branch yang SAMA -> push
MERGE : checks hijau + approval (mode 4: dua) -> squash merge
SELESAI: sync main -> hapus branch
        (perlu di release? cherry-pick -x)
```

Kesalahan yang harus dihindari

Kesalahan	Perbaikan
Coding langsung di main	Branch per task; cek <code>--show-current</code>
Menyalin potongan code	Selalu tulis file versi UTUH
<code>git add .</code> tanpa lihat diff	Stage hanya file relevan
PR baru saat request changes	Perbaiki branch yang sama
Asal pilih sisi conflict	Pahami dua maksud, gabungkan, test
Force push polos	<code>--force-with-lease</code> sesuai policy
Merge saat CI merah	Perbaiki checks dulu
Approve PR sendiri	Review silang / tunggu reviewer
Cherry-pick tanpa <code>-x</code>	Jejak asal commit hilang

Kriteria lulus

- [] SSH multi-akun jalan: `ssh -T <alias>` menyapa akun yang benar
- [] Satu siklus penuh APP-101: branch -> PR -> review -> squash merge
- [] Menangani request changes tanpa PR baru
- [] Menyelesaikan conflict merge DAN rebase, lalu test ulang
- [] Mode 3 akun: dua PR paralel + review silang + konflik alami selesai
- [] Mode 4 akun: 2 approval, paham stale review, merge oleh maintainer
- [] Cherry-pick hotfix ke release branch dengan `-x`

Kalau semua tercentang dan kamu bisa menjelaskan KENAPA setiap langkah dilakukan – bukan sekadar hafal – kamu siap masuk ke tim software sungguhan. Ulangi simulasi dengan rotasi role sampai

semua posisi pernah kamu rasakan.